

Database Systems: Fundamentals and Architecture

Based on Connolly & Begg (Chapters 1 & 2) and Lecture Slides

1. Introduction to Databases & DBMS (Chapter 1)

The Old Way: File-Based Systems

Before databases, companies used a **File-Based Approach**. Imagine a university where the HR department has its own set of Excel files for professors, and the Payroll department has a completely separate set of files for the exact same professors.

- **The Problem with File-Based Systems:**
 - * **Data Redundancy:** You are saving the exact same data in multiple places. It wastes storage space.
 - **Data Isolation:** The HR file doesn't "talk" to the Payroll file. If you want a report combining data from both, it's a nightmare.
 - **Data Dependence:** The physical files and the software programs reading them are tightly linked. If you change the file structure (like adding a "Phone Number" column), you have to rewrite the actual application code that reads it.

The New Way: The Database Approach

To fix the mess of file-based systems, the **Database** was invented.

- **Database:** A shared, massive collection of logically related data. It's designed to meet the information needs of an entire organization. Instead of everyone having their own little files, everyone shares one giant, highly organized pool of data.
- **DBMS (Database Management System):** The software system that lets you talk to the database. You can't just open a database like a Word document; you need a DBMS (like MySQL, Oracle, or SQL Server) to define, create, maintain, and secure the data.

Advantages & Disadvantages of a DBMS

Advantages (Why we love it)	Disadvantages (The trade-offs)
Control of Redundancy: Data is stored exactly once. No more duplicate files.	Complexity: A DBMS is a massive piece of software. It requires specialized knowledge to set up and run.
Data Consistency: Because data is stored once, if you update a student's address, it instantly updates for every department.	Cost: High costs for buying the software, upgrading hardware to run it, and hiring trained staff.

Data Sharing: Different departments can query the exact same pool of data in new, creative ways.	Performance Impact: Because it does so much background work (security, backups, checking rules), it can be slower than a simple flat file for basic tasks.
Better Security & Backup: Built-in, professional tools to protect data from hackers and hardware crashes.	Higher Impact of Failure: Because everything is centralized, if the central database goes down, <i>the whole company</i> stops working.

2. The Database Environment: Components & Roles (Chapter 1)

A database environment isn't just a piece of software; it's an entire ecosystem made of machines, rules, and humans.

The 5 Components of the DBMS Environment

1. **Hardware:** The physical stuff. The servers, hard drives, memory, and network cables running the system.
2. **Software:** This includes the DBMS itself, the computer's Operating System (like Linux or Windows), and the application programs (like a website) interacting with it.
3. **Data:** The most important part! It acts as the bridge between the machine components (hardware/software) and the human components.
4. **Procedures:** The official rules and instructions. For example, "How to log in," "How to run the weekly backup," or "What to do if the server crashes."
5. **People:** The humans who manage and use the system.

Key Roles (The People)

- **Data Administrator (DA):** The *business* boss of the data. They decide *what* data needs to be stored to help the company run.
- **Database Administrator (DBA):** The *technical* boss. They actually install the software, manage security passwords, and make sure the database runs fast without crashing.
- **Database Designers:**
 - *Logical Designer:* The architect. They draw the blueprint of what the tables will look like and how they connect, without worrying about computers.
 - *Physical Designer:* The builder. They take the blueprint and decide exactly *how* it will be stored on the physical hard drives (like setting up indexes for faster searching).
- **Application Developers:** The coders who write the software (like a mobile app or website) that talks to the database.
- **End Users:** The people actually using the app.

- *Naive Users*: Everyday people just clicking buttons (like you buying something on Amazon).
- *Sophisticated Users*: Data analysts or scientists who write their own complex SQL queries to find trends.

3. Database Languages & Functions (Chapter 2)

How do we actually give instructions to a database? We use specific languages, primarily **SQL (Structured Query Language)**, which is split into different categories based on what you want to do.

Database Languages

Language Type	What it stands for	Purpose	Real-World Analogy	Key Commands
DDL	Data Definition Language	Used to define the <i>structure</i> or blueprint of the database.	Building the actual house (walls, rooms).	CREATE, ALTER, DROP
DML	Data Manipulation Language	Used to read, insert, update, or delete the actual <i>data</i> inside the structures.	Moving furniture and people in and out of the house.	SELECT, INSERT, UPDATE, DELETE

Note: DML can be Procedural (you tell the system exactly HOW to get the data step-by-step, like programming) or Non-Procedural/Declarative (you just tell the system WHAT data you want, and it figures out the best way to get it. SQL is declarative).

Core Functions of a DBMS

A modern DBMS does a lot of heavy lifting behind the scenes. Its main jobs include:

1. **Data Storage, Retrieval, and Update:** The most basic function—saving data, finding it when asked, and changing it when needed.
2. **User-Accessible Catalog (Data Dictionary):** A built-in system that stores descriptions of the data itself (metadata). It remembers that "Phone_Number" is a 10-digit number.
3. **Transaction Support:** Ensures that complex actions happen completely or not at all. (e.g., If you transfer \$50 to a friend, the system deducts \$50 from you AND adds \$50 to them. If the internet crashes halfway, it cancels the whole thing so your money isn't lost).
4. **Concurrency Control Services:** Ensures that if two people try to book the exact same

concert ticket at the exact same millisecond, the system handles it fairly and doesn't double-book.

5. **Recovery & Security Services:** Auto-recovering data if a hard drive fails, and requiring usernames/passwords so hackers can't get in.

4. Database Architectures & Data Independence (Chapter 2)

The ANSI-SPARC Three-Level Architecture

This is a massive concept in databases. It's a framework designed to separate the user's view of the data from the complex physical storage details. Think of it like a restaurant.

1. **External Level (The User's View):** * *Analogy:* The restaurant menu.
 - *Details:* This is what the end-user sees. An HR manager sees a screen with employee names and salaries. They don't see the IT department's data. It simplifies things for the user and adds security.
2. **Conceptual Level (The Community View):** * *Analogy:* The Head Chef's master recipe book and kitchen blueprint.
 - *Details:* This is the logical structure of the *entire* database as seen by the DBA. It defines all the tables, data types, and relationships. It completely hides how data is physically stored on the computer.
3. **Internal Level (The Physical View):** * *Analogy:* How the ingredients are actually organized in the deep freezer and pantry.
 - *Details:* How the data is physically stored on the computer hardware. It deals with bytes, hard disk tracks, data compression, and physical indexes.

Data Independence

Because we split the database into these 3 levels, we get **Data Independence**—the ability to change the design at one level without breaking the level above it. This saves companies millions of dollars in software rewrites.

- **Logical Data Independence:** You can change the Conceptual Level (like adding a new "Bonus" column to the Employee table) without breaking the External Level (the HR manager's app still works exactly as it did before; it just ignores the new column).
- **Physical Data Independence:** You can change the Internal Level (like buying a faster hard drive and changing how files are compressed) without having to rewrite any of the database tables at the Conceptual Level.

Multi-Tier Architectures (Client-Server)

How are applications and databases physically set up over networks?

- **Teleprocessing / Mainframe:** The old-school way. One massive, super-expensive computer does *all* the processing. Users just have "dumb terminals" (a keyboard and a monitor with no brain) to type into it.

- **Two-Tier Client-Server:** The workload is split in two. The user's computer (Client) runs the application and the user interface. It talks directly across the network to the Database Server, which holds the data.
- **Three-Tier Client-Server:** The modern standard (this is how the internet works).
 1. *Tier 1 (Client):* Your web browser or mobile app (Handles the User Interface).
 2. *Tier 2 (Application Server):* A middle server that holds the "business logic" (the brain of the app that calculates things).
 3. *Tier 3 (Database Server):* Holds the data.
 - *Advantage:* Highly scalable and secure. The user's phone never talks directly to the database; it has to go through the middle application server first.

5. Integrity Constraints (Lecture 6 Add-On)

To keep a database accurate and reliable, we apply **Integrity Constraints**. These are strict rules the DBMS enforces so users can't enter garbage data.

1. **Domain Constraints:** * Restricts the type of data that can go into a column.
 - *Example:* You can't put the word "Hello" in an Age column. The domain is strictly numbers.
2. **Entity Integrity Constraint:** * Every table must have a Primary Key (a unique identifier like a Student ID), and this Primary Key can **never be NULL** (empty).
 - *Example:* You can't enroll a student in the system without giving them a Student ID.
3. **Referential Integrity Constraint:** * This involves Foreign Keys. If Table A references Table B, the thing it's referencing must actually exist.
 - *Example:* You cannot assign a student to "Department 99" if Department 99 does not exist in the Departments table.
4. **Key Constraint:** * Ensures that certain columns have completely unique values.
 - *Example:* Two people can have the same First Name, but they cannot have the same National ID number or Email Address.